Linear Image Filters

Shree K. Nayar

Columbia University

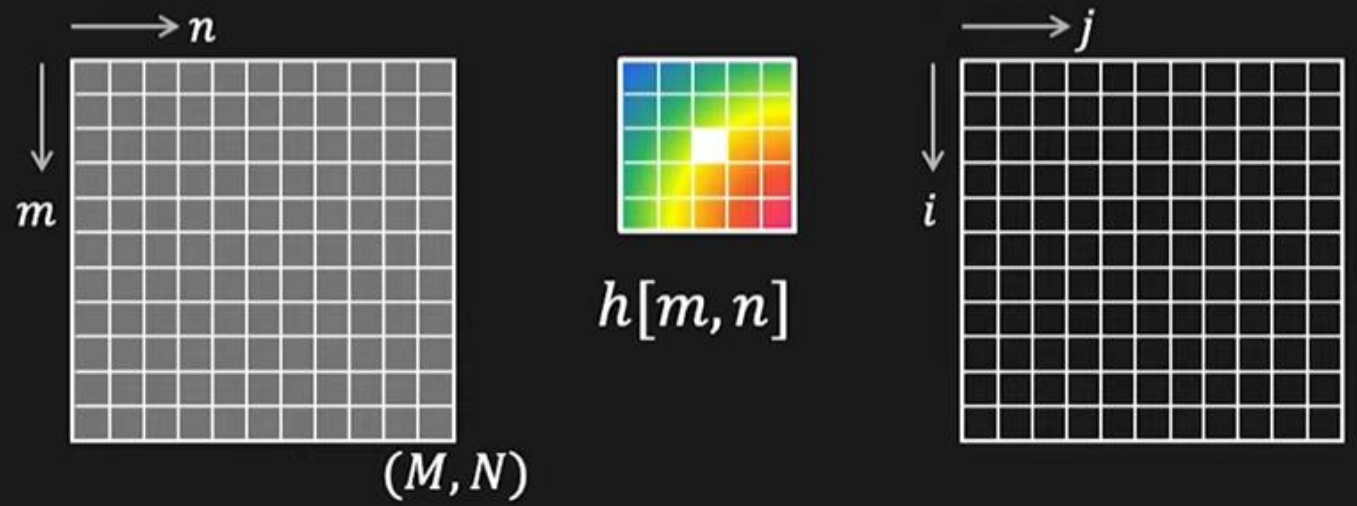
Topic: Image Processing I, Module: Imaging

First Principles of Computer Vision

Convolution with Discrete Images



$$g[i,j] = \sum_{m=1}^{M} \sum_{n=1}^{N} f[m,n]h[i-m,j-n]$$



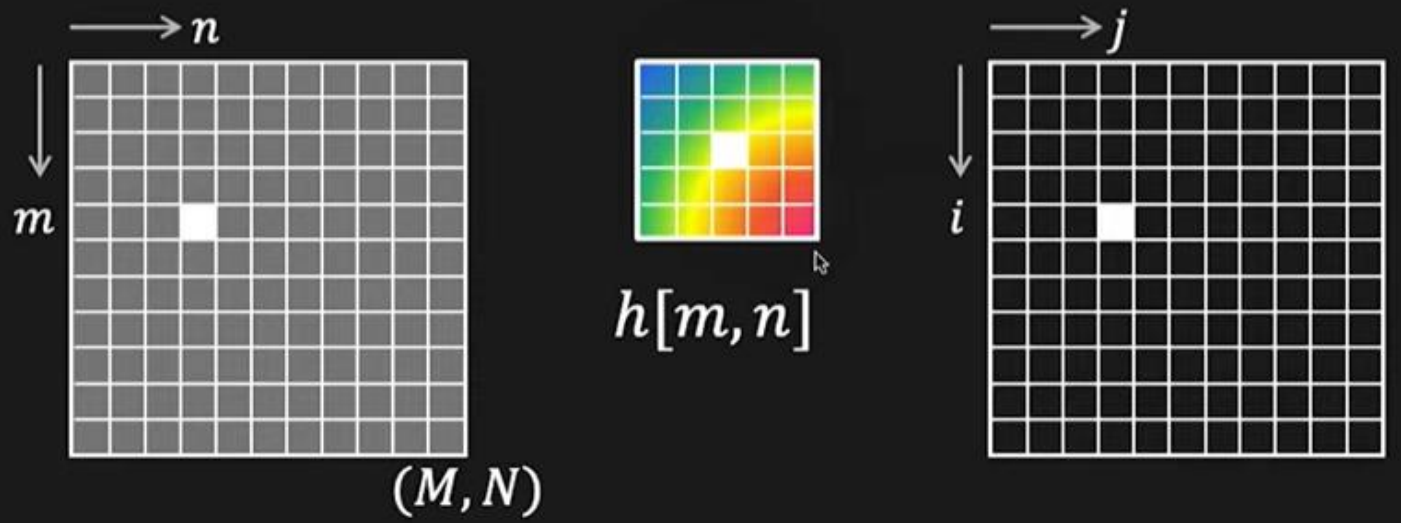
© 2020 Shree K. Nayar

Convolution with Discrete Images



$$g[i,j] = \sum_{m=1}^{M} \sum_{n=1}^{N} f[m,n]h[i-m,j-n]$$

"Mask," "Kernel," "Filter"

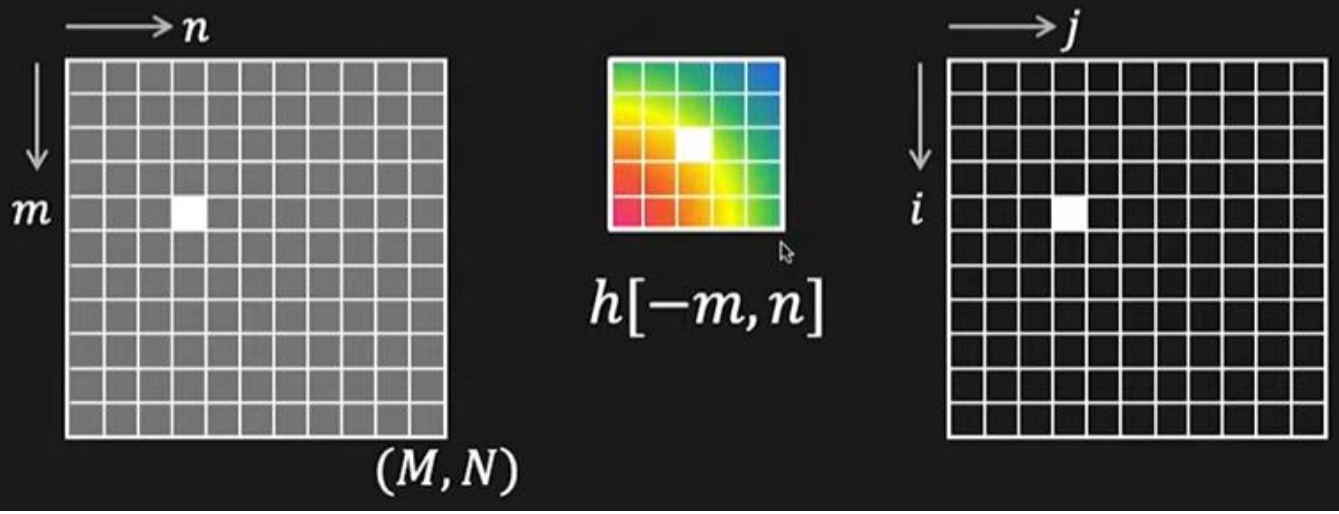


Convolution with Discrete Images

$$f[i,j] \longrightarrow \boxed{h[i,j]} \longrightarrow g[i,j]$$

$$g[i,j] = \sum_{m=1}^{M} \sum_{n=1}^{N} f[m,n]h[i-m,j-n]$$

"Mask," "Kernel," "Filter"

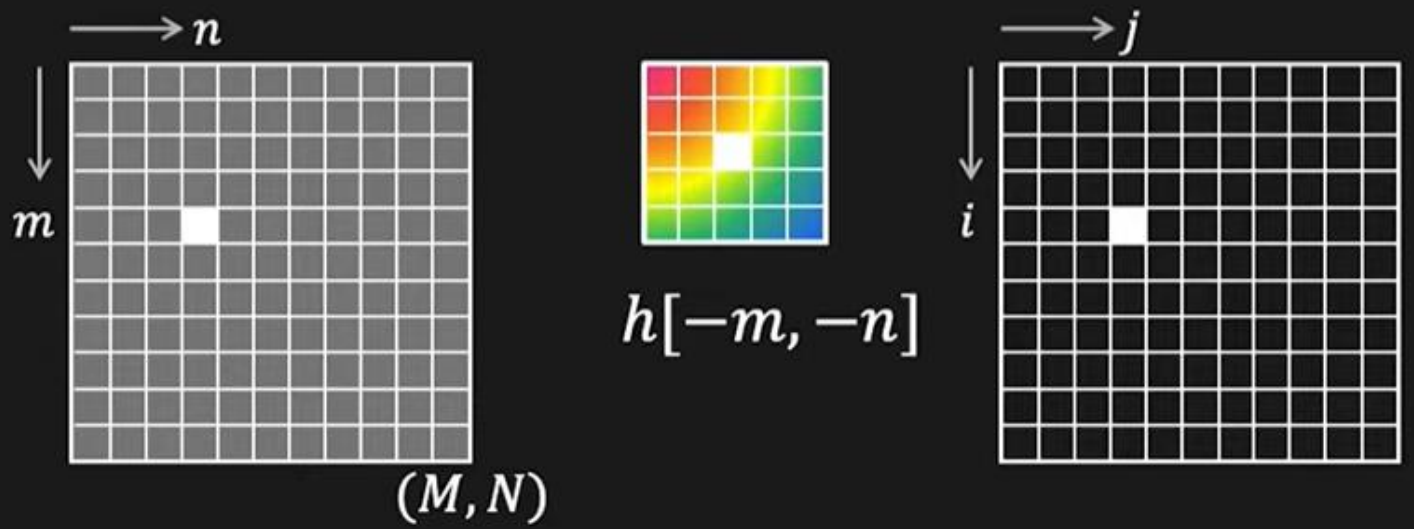


Convolution with Discrete Images



$$g[i,j] = \sum_{m=1}^{M} \sum_{n=1}^{N} f[m,n]h[i-m,j-n]$$

"Mask," "Kernel," "Filter"

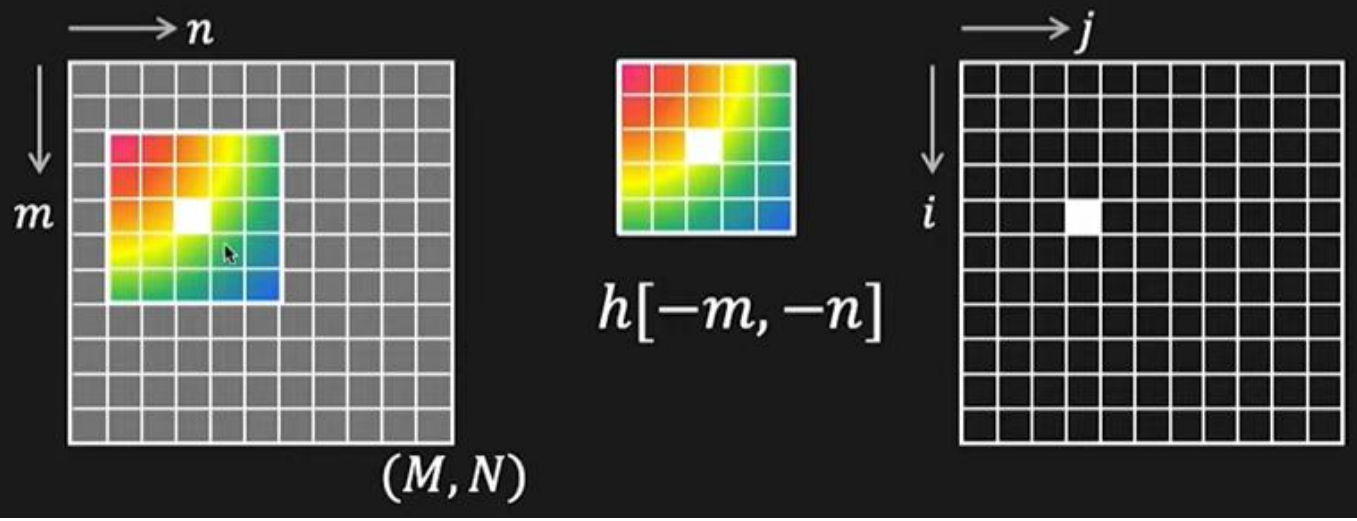


Convolution with Discrete Images

$$f[i,j] \longrightarrow \boxed{h[i,j]} \longrightarrow g[i,j]$$

$$g[i,j] = \sum_{m=1}^{M} \sum_{n=1}^{N} f[m,n]h[i-m, j-n]$$

"Mask," "Kernel," "Filter"

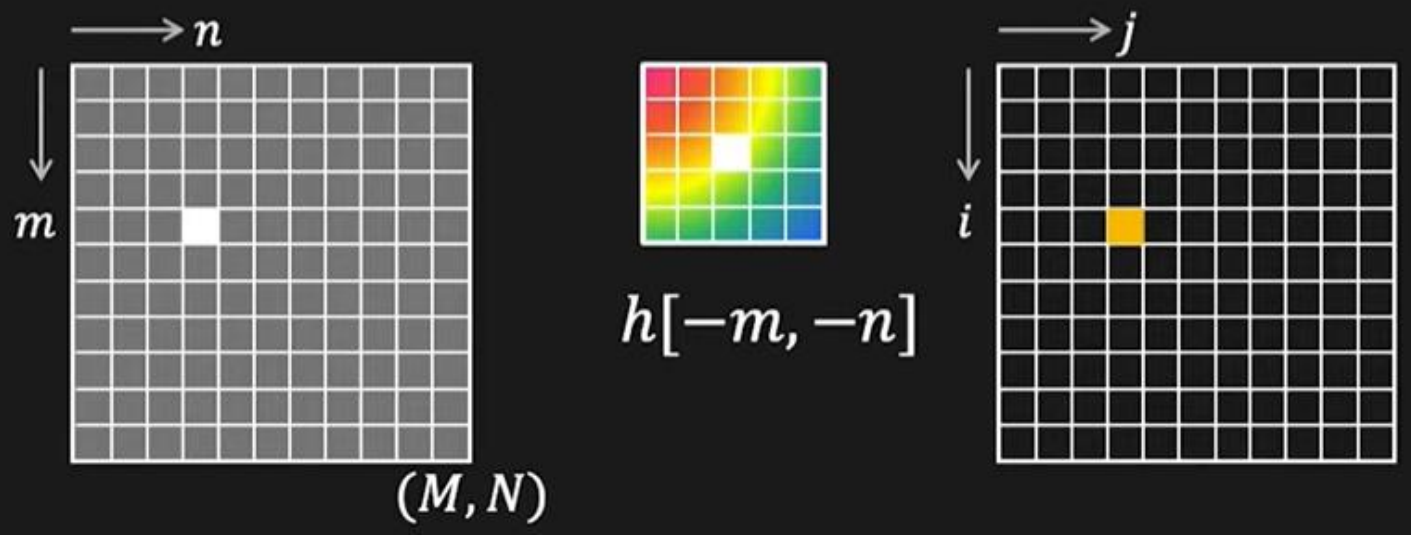


Convolution with Discrete Images

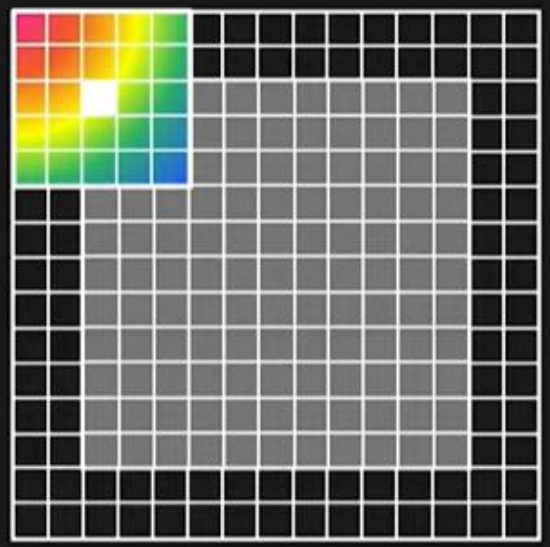
$$f[i,j] \longrightarrow \boxed{h[i,j]} \longrightarrow g[i,j]$$

$$g[i,j] = \sum_{m=1}^{M} \sum_{n=1}^{N} f[m,n]h[i-m,j-n]$$

"Mask," "Kernel," "Filter"



Border Problem



Solution:

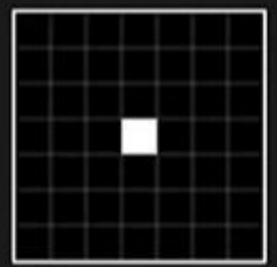
- Ignore border
- Pad with constant value
- Pad with reflection

Example: Impulse Filter

Input



$*$



$=$

Output



$f(x, y)$ $\delta(x, y)$ $f(x, y)$

Example: Image Shift

Input



$f(x, y)$

$*$



$\delta(x - u, y - v)$

$=$

Output



$f(x - u, y - v)$

Example: Averaging

Input

Output



$*$



$=$

"Box Filter"
5 x 5

$f(x, y)$

$a(x, y)$

Example: Averaging

Input



*



"Box Filter"
5 x 5

=

Output



$f(x, y)$ $a(x, y)$ $g(x, y)$

Result Image is saturated. Why?

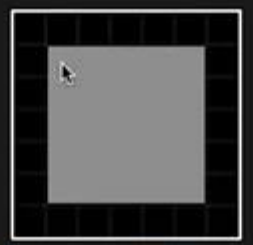
Example: Averaging

Input



$f(x, y)$

$*$



"Box Filter"
5 x 5

$a(x, y)$

$=$

Output



$g(x, y)$

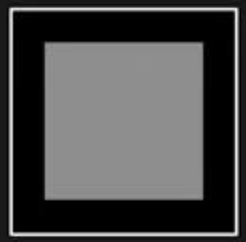
Sum of all the filter (kernel) weights should be 1.

Smoothing With Box Filter

Input



$*$



"Box Filter"
21 x 21

$=$

Output



$f(x, y)$ $a(x, y)$ $g(x, y)$

Image smoothed with a box filter does not look "natural." Has blocky artifacts.

Shree K. Nayar

Smoothing With "Fuzzy" Filter

Input



$f(x,y)$

*



"Fuzzy Filter"
21 x 21

$b(x,y)$

=

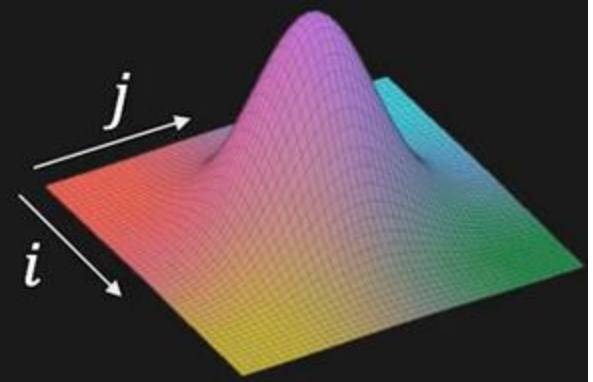
Output



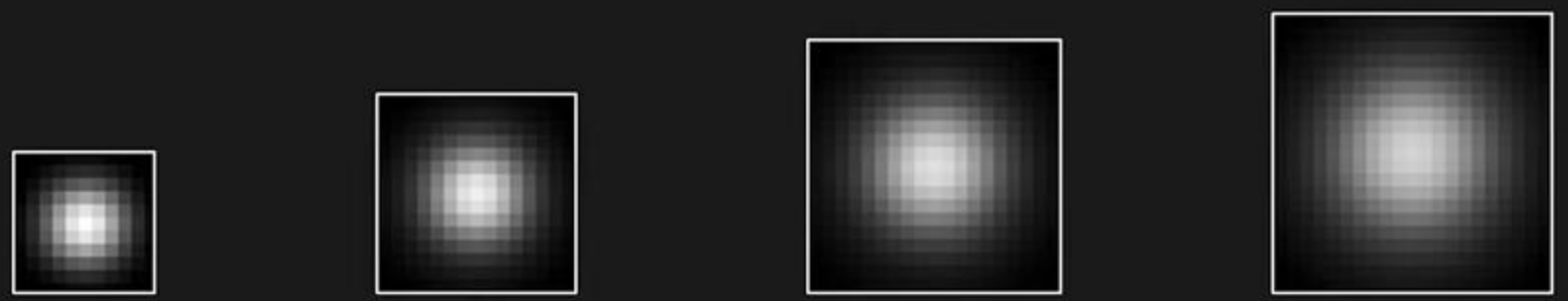
$g(x,y)$

Gaussian Kernel: A Fuzzy Filter

$$n_\sigma[i,j] = \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2}\left(\frac{i^2+j^2}{\sigma^2}\right)}$$



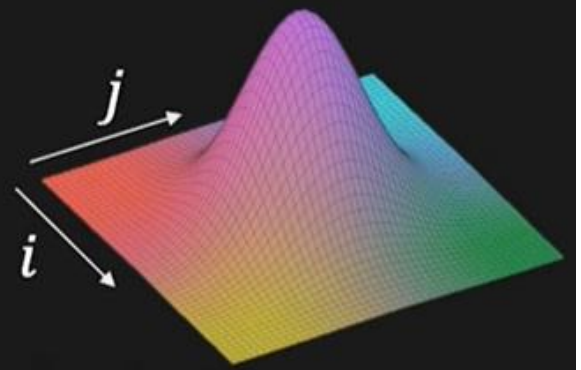
σ^2: Variance



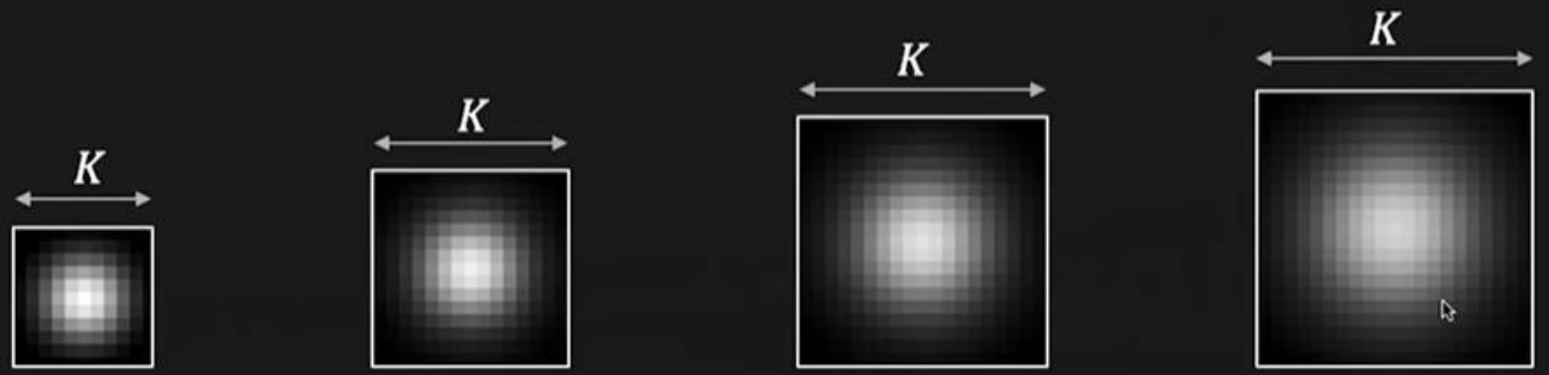
$\sigma = 2$ $\sigma = 3$ $\sigma = 4$ $\sigma = 5$

Gaussian Kernel: A Fuzzy Filter

$$n_\sigma[i,j] = \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2}\left(\frac{i^2+j^2}{\sigma^2}\right)}$$



σ^2: Variance



$\sigma = 2$ $\sigma = 3$ $\sigma = 4$ $\sigma = 5$

Rule of thumb: Set kernel size $K \approx 2\pi\sigma$

Gaussian Smoothing

Input



$*$



$\sigma = 4$

$=$

Output



$f(x, y)$ $n_4(x, y)$ $g(x, y)$

Larger the kernel (or σ), more the blurring

Gaussian Smoothing

Input



$f(x, y)$

$*$



$\sigma = 16$

$n_{16}(x, y)$

$=$

Output



$g(x, y)$

Larger the kernel (or σ), more the blurring

2020 Shree K. Nayar

Gaussian Smoothing is Separable

$$g[i,j] = \frac{1}{2\pi\sigma^2} \sum_{m=1}^{K} \sum_{n=1}^{K} e^{-\frac{1}{2}\left(\frac{m^2+n^2}{\sigma^2}\right)} f[i-m, j-n]$$

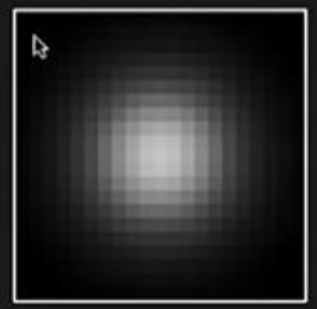
$$g[i,j] = \frac{1}{2\pi\sigma^2} \sum_{m=1}^{K} e^{-\frac{1}{2}\left(\frac{m^2}{\sigma^2}\right)} \cdot \sum_{n=1}^{K} e^{-\frac{1}{2}\left(\frac{n^2}{\sigma^2}\right)} f[i-m, j-n]$$

Gaussian Smoothing is Separable

$$g[i,j] = \frac{1}{2\pi\sigma^2} \sum_{m=1}^{K} \sum_{n=1}^{K} e^{-\frac{1}{2}\left(\frac{m^2+n^2}{\sigma^2}\right)} f[i-m, j-n]$$

$$g[i,j] = \frac{1}{2\pi\sigma^2} \sum_{m=1}^{K} e^{-\frac{1}{2}\left(\frac{m^2}{\sigma^2}\right)} \cdot \sum_{n=1}^{K} e^{-\frac{1}{2}\left(\frac{n^2}{\sigma^2}\right)} f[i-m, j-n]$$

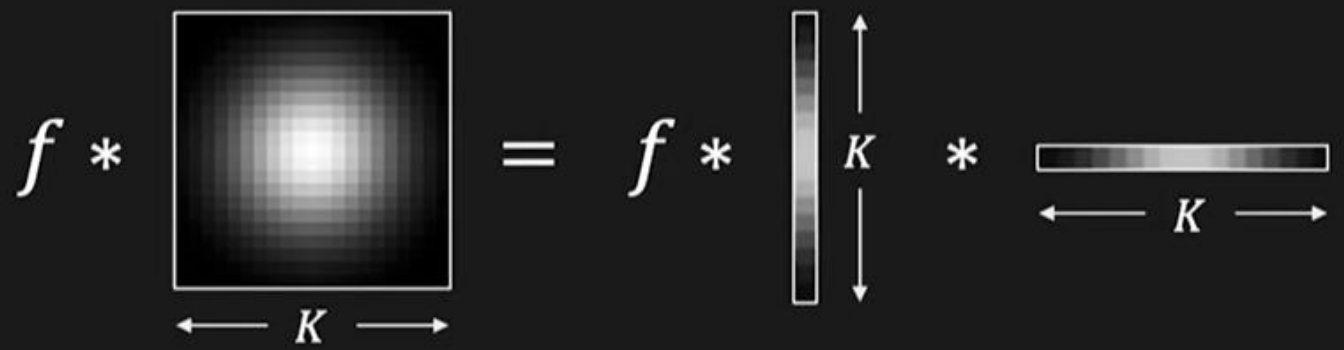
Using One 2D Gaussian Filter ≡ Using Two 1D Gaussian Filters

$f *$  $=$ $f *$  $*$ 

020 Shree K. Nayar

Gaussian Smoothing is Separable

Using One 2D Gaussian Filter ≡ Using Two 1D Gaussian Filters



Which one is faster? Why?

K^2 Multiplications	$2K$ Multiplications
$K^2 - 1$ Additions	$2(K-1)$ Additions

Non-Linear Image Filters

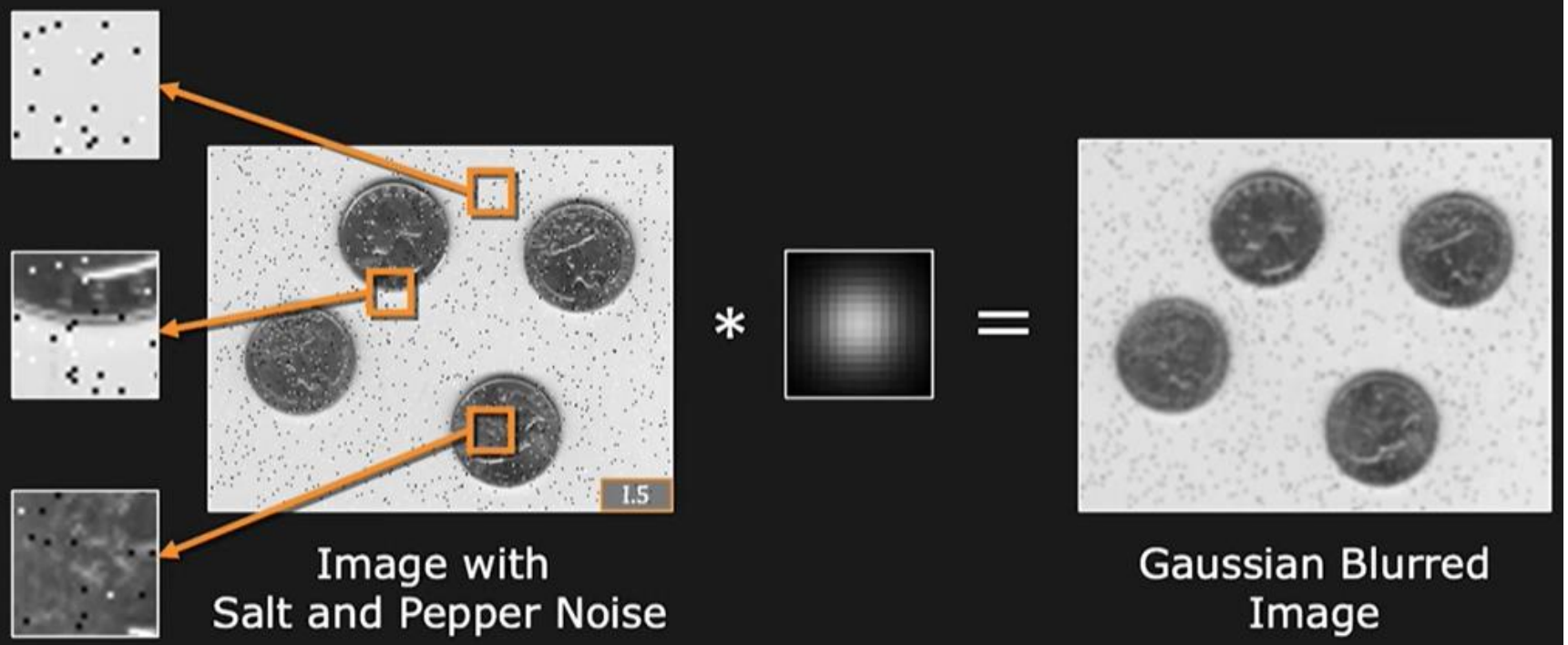
Shree K. Nayar

Columbia University

Topic: Image Processing I, Module: Imaging

First Principles of Computer Vision

Smoothing to Remove Image Noise



Problem with Smoothing:

- Does not remove outliers (Noise)
- Smooths edges (Blur)

2020 Shree K. Nayar

Median Filtering

1. Sort the K^2 values in window centered at the pixel
2. Assign the Middle Value (Median) to pixel



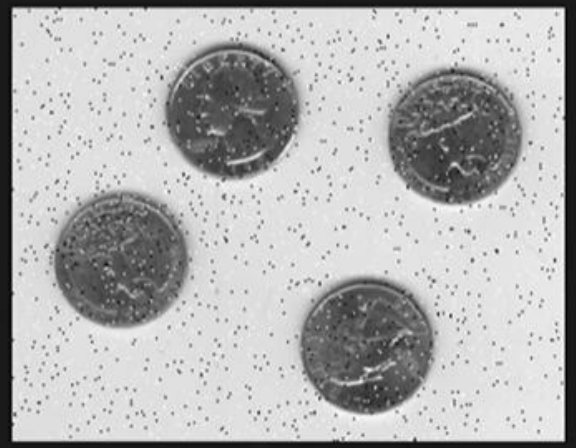


Image with
Salt and Pepper Noise



Median Filtered
Image ($K = 3$)

2020 Shree K. Nayar

Median Filtering

Not Effective when Image Noise is not a Simple Salt and Pepper Noise.

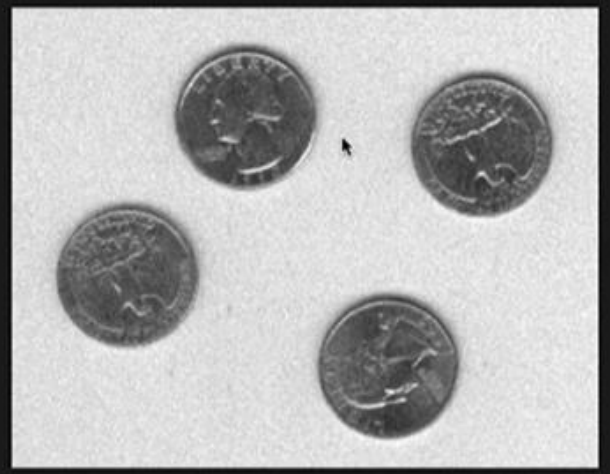
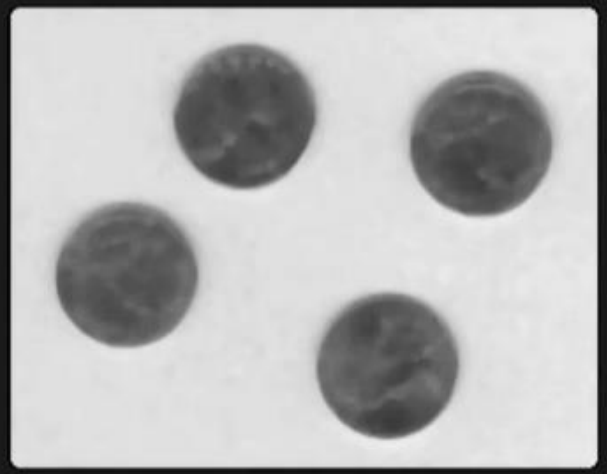


Image with Noise



Median Filtered Image ($K = 7$)

Larger K causes blurring of image detail

Median Filtering

Not Effective when Image Noise is not a Simple Salt and Pepper Noise.

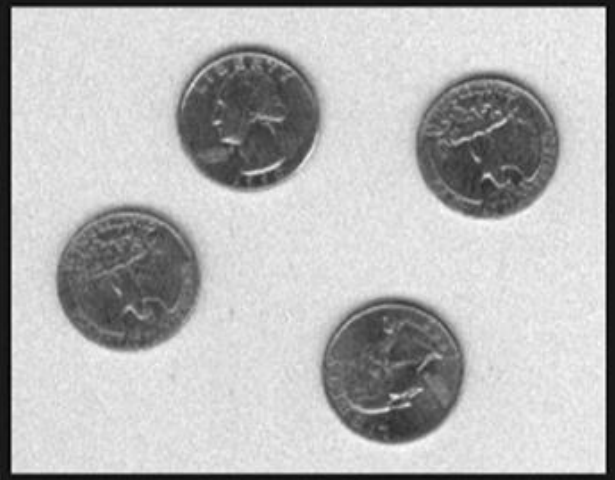
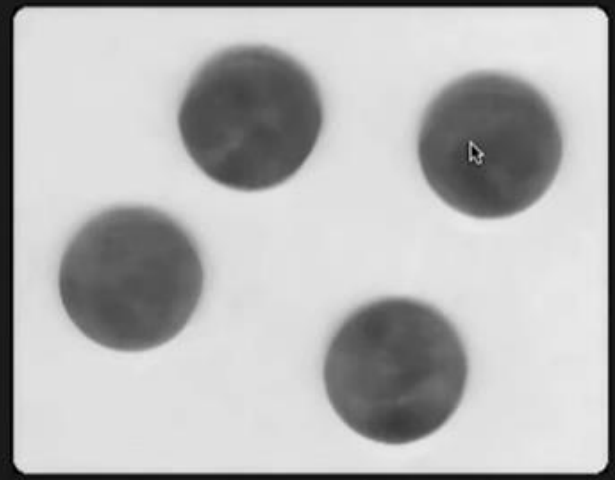


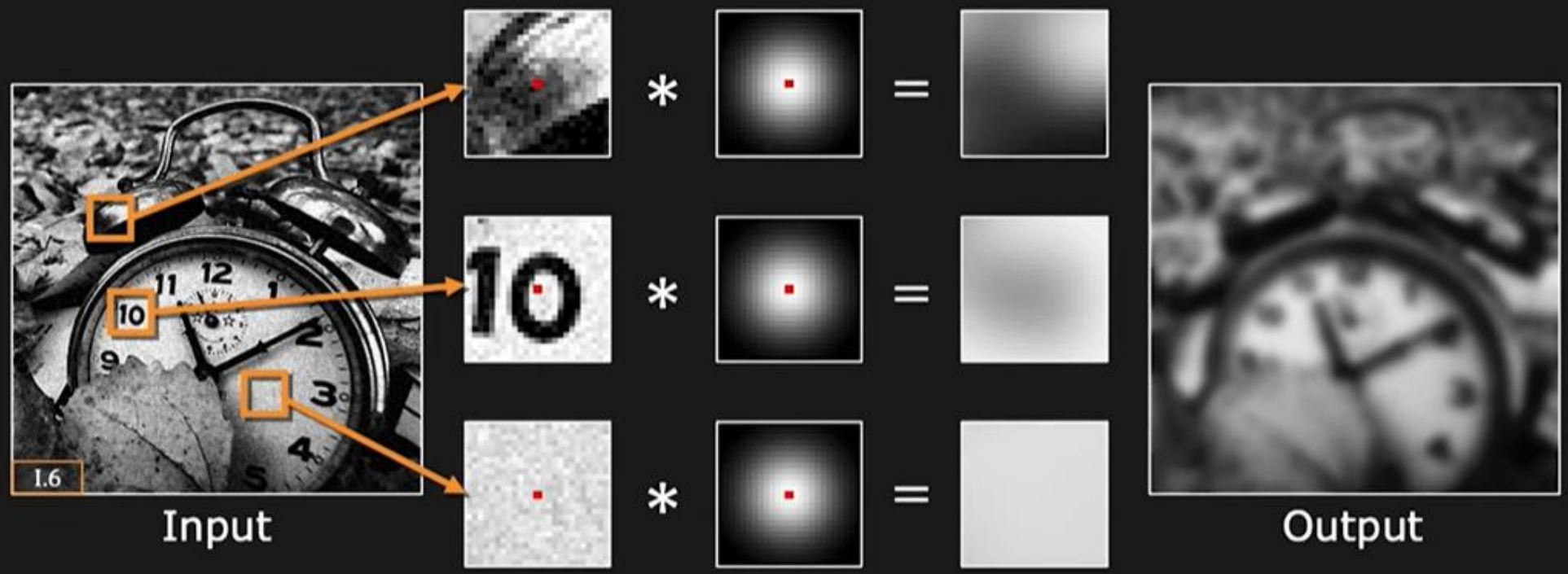
Image with Noise



Median Filtered Image ($K = 11$)

Larger K causes blurring of image detail

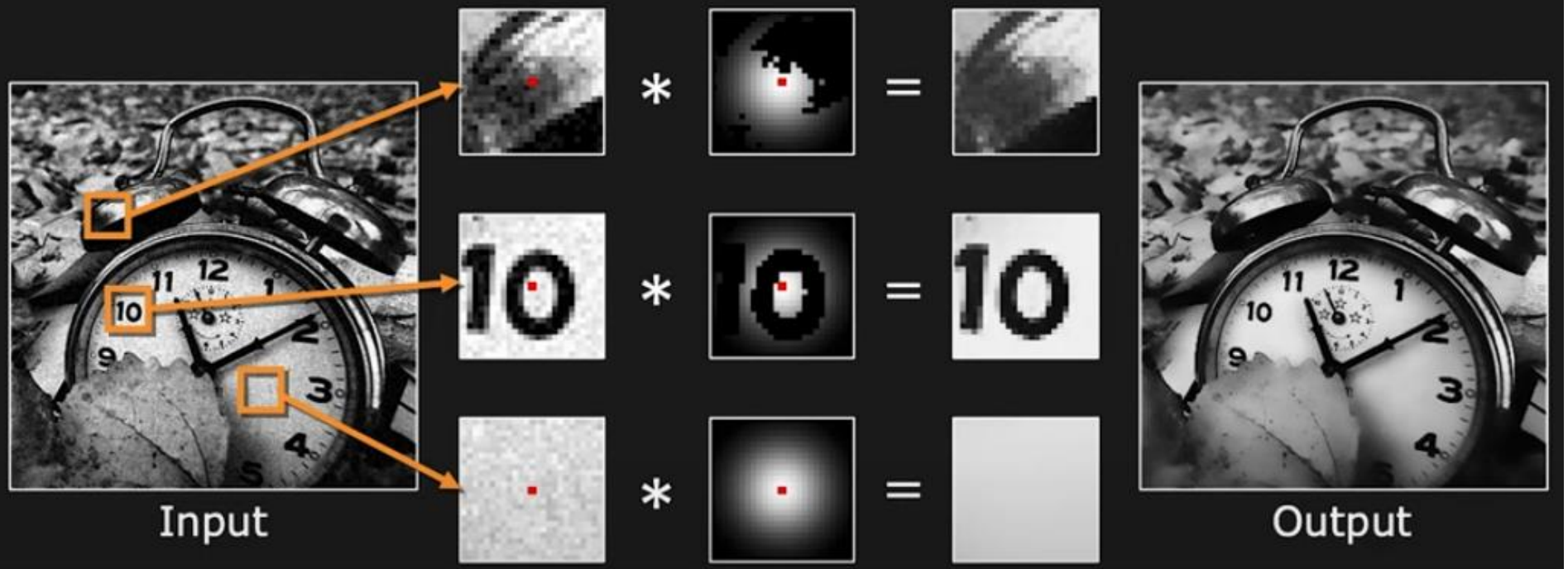
Revisiting Gaussian Smoothing



Same Gaussian kernel is used everywhere.

Blurs across edges.

Blur Similar Pixels Only

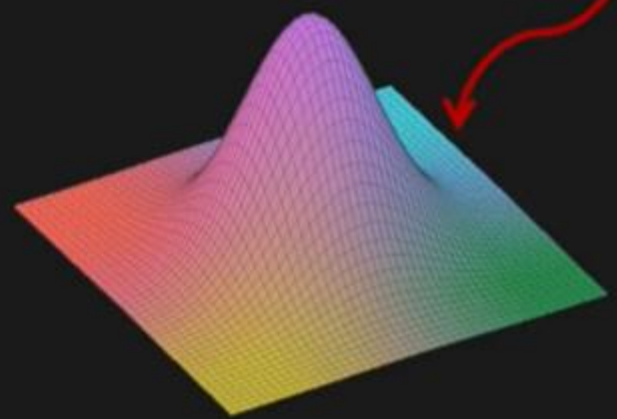


"Bias" Gaussian Kernel such that pixels not similar in intensity to the center pixel receive a lower weight.

Bilateral Filter: Start With Gaussian

Spatial Gaussian

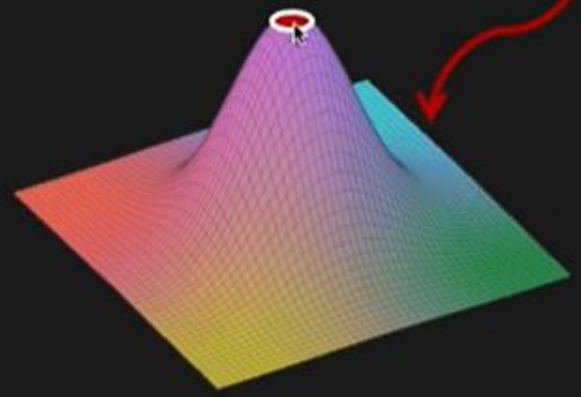
$$g[i,j] = \frac{1}{W_s}\sum_m\sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]$$

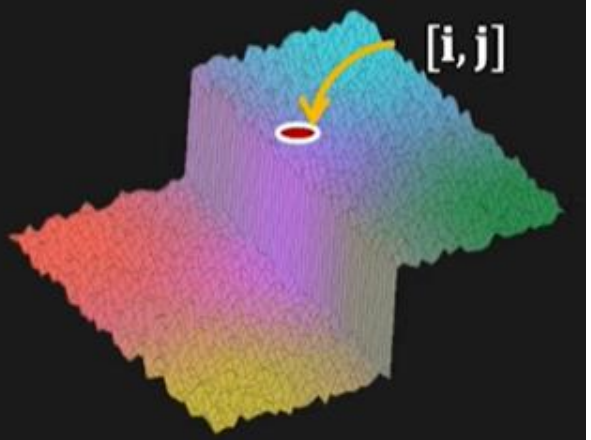


Bilateral Filter: Start With Gaussian

Spatial Gaussian

$$g[i,j] = \frac{1}{W_s} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]$$



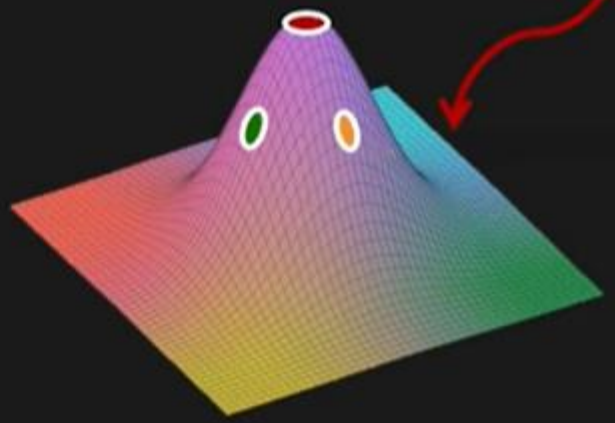


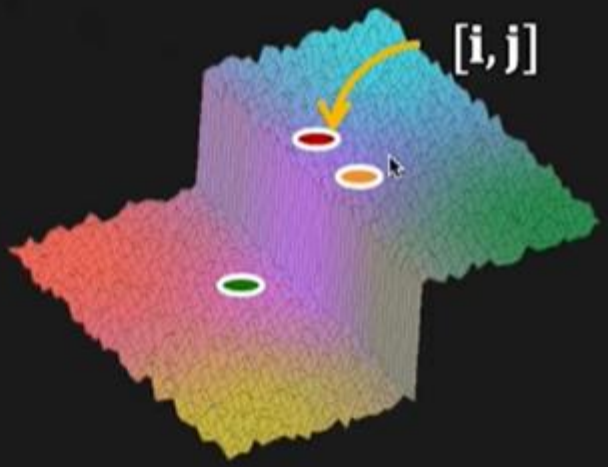
Input (f)

Bilateral Filter: Start With Gaussian

Spatial Gaussian

$$g[i,j] = \frac{1}{W_s} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]$$



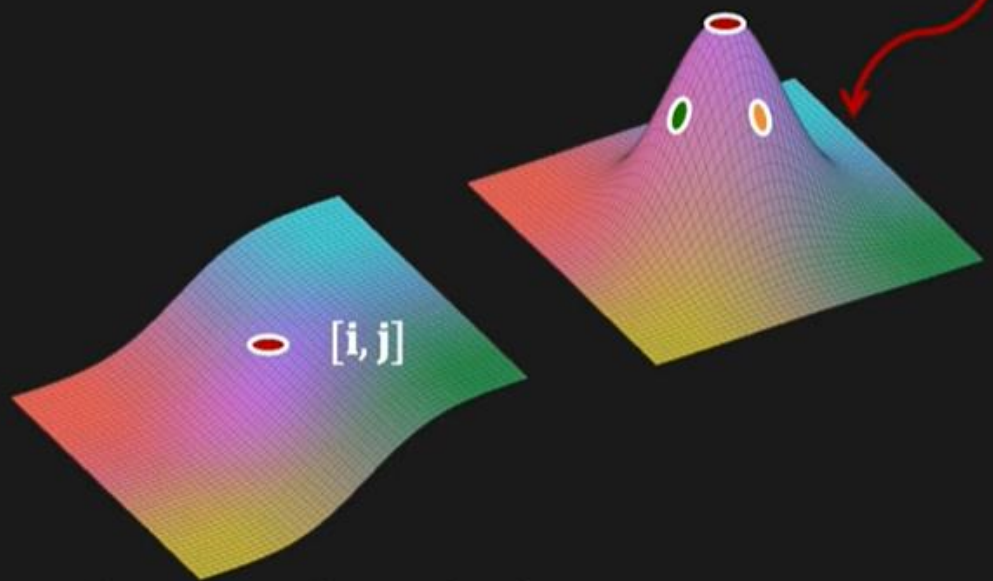


Input (f)

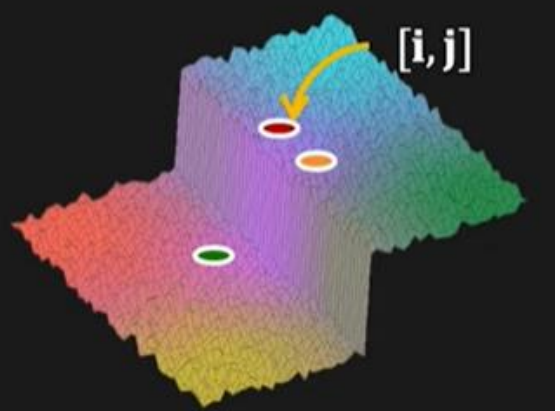
Bilateral Filter: Start With Gaussian

Spatial Gaussian

$$g[i,j] = \frac{1}{W_s} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]$$



Gaussian Smoothed Output (g)



Input (f)

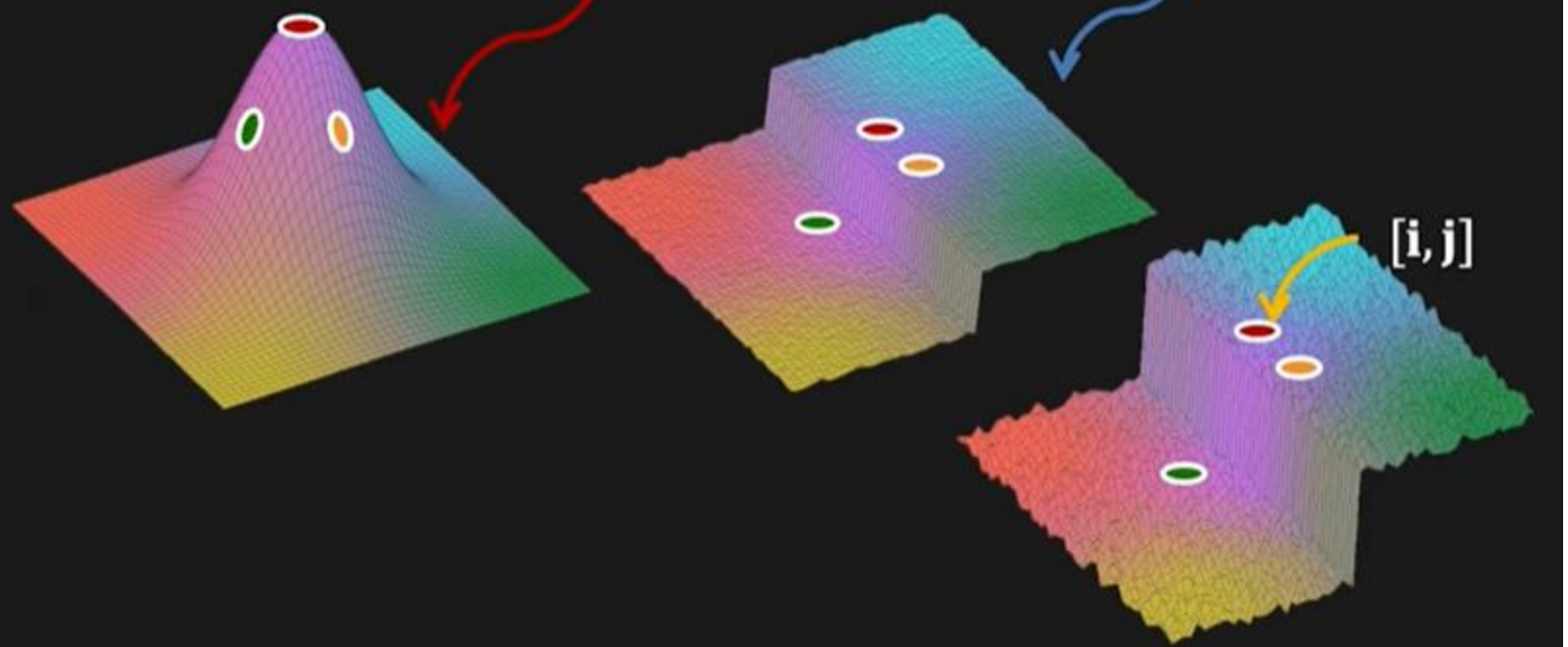
Gaussian blurs across edges

© 2020 Shree K. Nayar

Bilateral Filter: Add Bias to Gaussian

Spatial Gaussian Brightness Gaussian

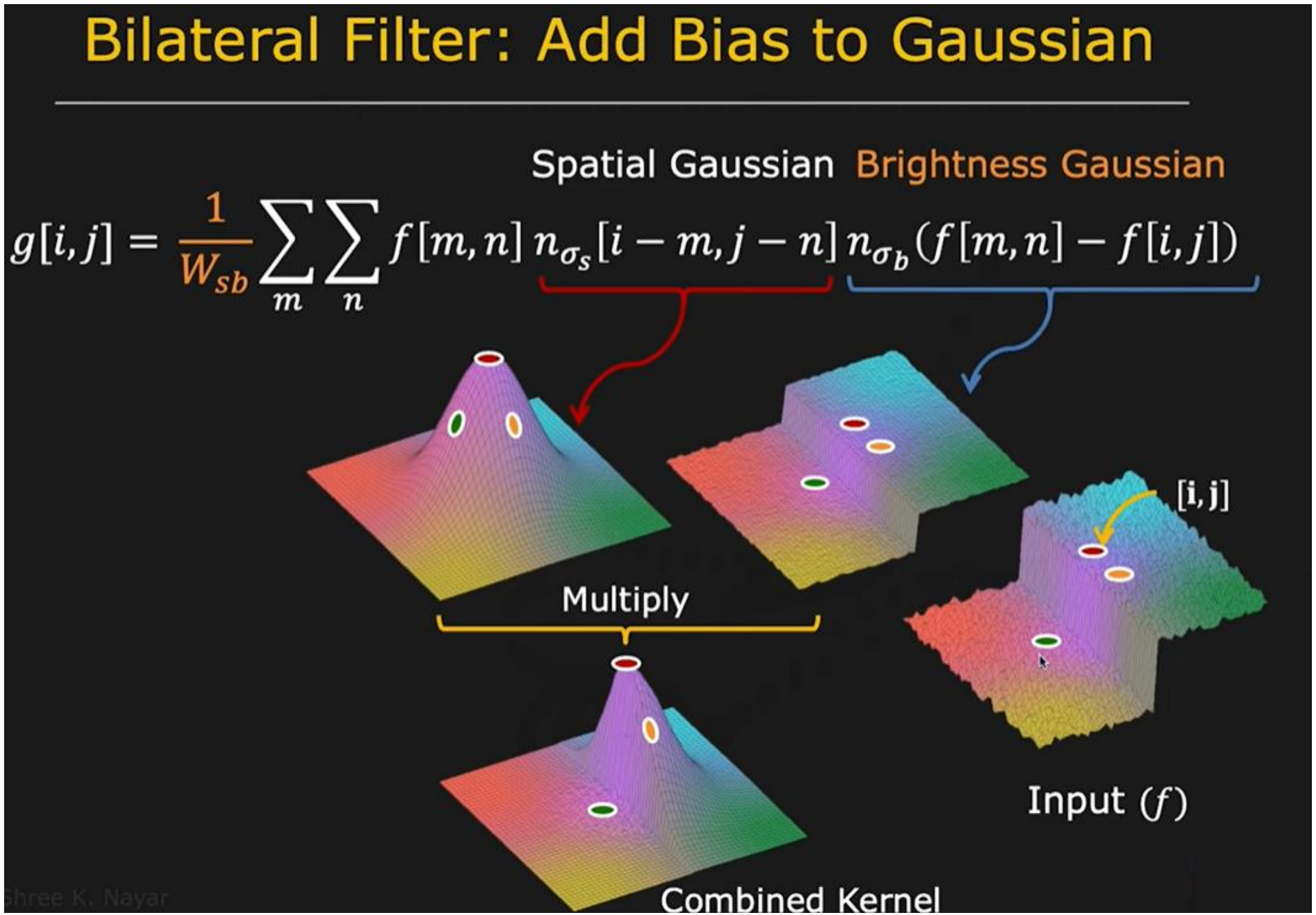
$$g[i,j] = \frac{1}{W_{sb}} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]\, n_{\sigma_b}(f[m,n] - f[i,j])$$



Bilateral Filter: Add Bias to Gaussian

Spatial Gaussian Brightness Gaussian

$$g[i,j] = \frac{1}{W_{sb}} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]\, n_{\sigma_b}(f[m,n] - f[i,j])$$

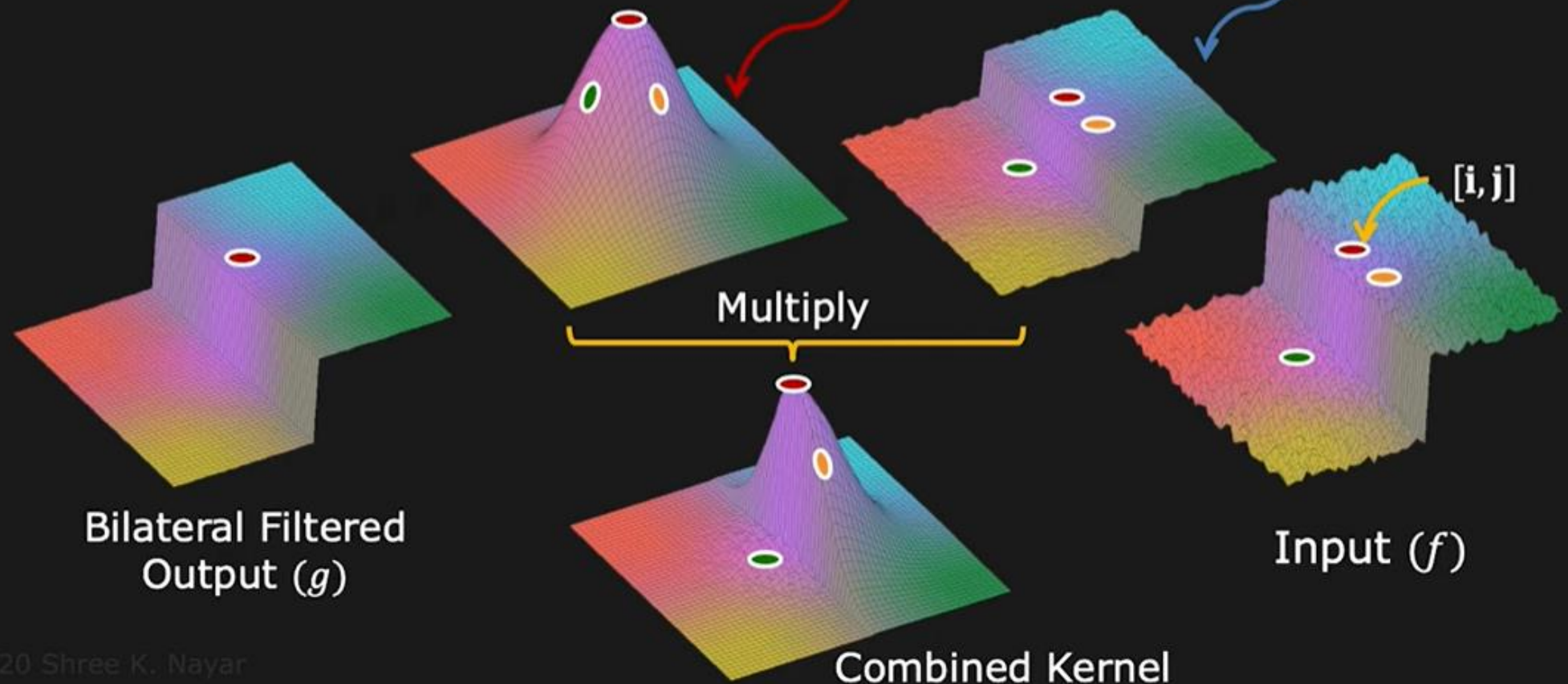


Shree K. Nayar

Bilateral Filter: Add Bias to Gaussian

Spatial Gaussian Brightness Gaussian

$$g[i,j] = \frac{1}{W_{sb}} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]\, n_{\sigma_b}(f[m,n] - f[i,j])$$



2020 Shree K. Nayar

Bilateral Filter: Summary

$$g[i,j] = \frac{1}{W_{sb}} \sum_m \sum_n f[m,n]\, n_{\sigma_s}[i-m, j-n]\, n_{\sigma_b}(f[m,n] - f[i,j])$$

Where:

$$n_{\sigma_s}[m,n] = \frac{1}{2\pi {\sigma_s}^2} e^{-\frac{1}{2}\left(\frac{m^2+n^2}{\sigma_s^2}\right)} \qquad n_{\sigma_b}(k) = \frac{1}{\sqrt{2\pi}\sigma_b} e^{-\frac{1}{2}\left(\frac{k^2}{\sigma_b^2}\right)}$$

$$W_{sb} = \sum_m \sum_n n_{\sigma_s}[i-m, j-n] n_{\sigma_b}(f[m,n] - f[i,j])$$

Non-linear Operation

(Cannot be implemented using convolution)

Gaussian vs. Bilateral Filtering: Example



Original



Gaussian
$\sigma_s = 2$



Bilateral
$\sigma_s = 2, \sigma_b = 10$

Gaussian vs. Bilateral Filtering: Example



Original



Gaussian
$\sigma_s = 8$



Bilateral
$\sigma_s = 8, \sigma_b = 10$

Bilateral Filtering: Changing σ_b



Bilateral
$\sigma_s = 6, \sigma_b = 10$



Bilateral
$\sigma_s = 6, \sigma_b = 20$



Bilateral
$\sigma_s = 6, \sigma_b = \infty$

References

- https://www.youtube.com/watch?v=-LD9MxBUFQo&list=PL2zRqk16wsdorCSZ5GWZQr1EMWXs2TDeu&index=4